

Laboratoire 3

Base de données PostgreSQL et pattern DAO

Suite directe du Laboratoire 2 — même projet, même équipe

Cours	Remise du code	Notation
420-930-MA (Été 2026)	Dimanche 20 septembre 2026, 23h59	Voir barème (section 9)
Équipe	Présentations orales	Où
La même qu'au Lab 2	Lundi 21 et mercredi 23 septembre	Teams — canal du groupe

Vous ne repartez pas de zéro. Le Laboratoire 3 se greffe sur le projet que vous avez remis le 11 septembre. Vous travaillez dans le **même dépôt GitHub**, avec la **même équipe**, sur le **même sujet**.

L'objectif du Lab 3 tient en une phrase : **remplacer le fichier CSV par une vraie base de données PostgreSQL, sans rien casser.**

1. Ce qui change par rapport à l'annonce du Lab 2

Les consignes du Laboratoire 2 annonçaient pour le Lab 3 : PostgreSQL, authentification, multithreading, écran de statistiques et API externe. **Le tronc obligatoire a été volontairement réduit.**

Fonctionnalité annoncée au Lab 2	Statut au Lab 3
Migration CSV → PostgreSQL (pattern DAO)	OBLIGATOIRE
Opérations CRUD depuis l'interface	OBLIGATOIRE
Authentification (login / mots de passe hashés)	Bonus
Wishlist / favoris persistants par utilisateur	Bonus
Multithreading (chargement asynchrone)	Bonus
Écran de statistiques avec charts	Bonus
Intégration d'une API externe	Bonus

Pourquoi cette réduction ? Vous disposez d'une **seule semaine** entre la remise du Lab 2 et celle du Lab 3, et cette semaine se termine par une présentation orale. Mieux vaut une couche d'accès aux données **irréprochable** qu'un empilement de fonctionnalités bâclées.

Le pattern DAO est une compétence attendue en entreprise : c'est exactement ce que vous retrouverez derrière Spring Data, Hibernate ou JPA. Le faire à la main une fois, proprement, vaut mieux que de le survoler.

Les bonus ne rapportent des points que si **tout le tronc obligatoire est terminé et fonctionnel**. Un projet avec une authentification et un DAO incomplet sera noté plus sévèrement qu'un projet sans authentification et au DAO parfait.

2. Objectifs pédagogiques

À la fin du Laboratoire 3, votre équipe saura :

- Concevoir un **schéma relationnel** à partir d'un modèle objet existant
- Écrire un **script SQL** de création de base (DDL) et le peupler

- Établir une **connexion JDBC** vers PostgreSQL depuis Java
- Implémenter le **pattern DAO** (Data Access Object) derrière une interface
- Écrire des requêtes **paramétrées** (PreparedStatement) et comprendre pourquoi
- Gérer proprement les ressources (**try-with-resources**) et les **SQLException**
- **Substituer une source de données à une autre** sans réécrire l'application

Le vrai test de votre Lab 2. Si vous avez respecté la **Règle 4** des consignes du Lab 2 (« programmez contre des interfaces », avec une interface **SourceDonnees** implémentée par **LecteurCSV**), alors brancher PostgreSQL consiste à écrire une nouvelle implémentation et à changer **une seule ligne** dans votre application.

Si vous ne l'avez pas fait, ce laboratoire commence par une étape de refactoring. C'est la leçon que ce laboratoire est conçu pour vous faire vivre.

3. Environnement technique

3.1 Ce qu'il faut installer

Outil	Version	Rôle
PostgreSQL	14 ou supérieur	Le serveur de base de données
pgAdmin 4	fourni avec PostgreSQL	Interface graphique pour créer et inspecter la base
Pilote JDBC PostgreSQL	42.7.x	Dépendance Maven qui permet à Java de parler à PostgreSQL

Le pilote s'ajoute dans votre `pom.xml` comme n'importe quelle dépendance Maven. Ses coordonnées sont :

groupId	artifactId
org.postgresql	postgresql

Attention au système de modules. Votre projet utilise `module-info.java` depuis le Lab 2. Le pilote JDBC doit y être déclaré, sinon la connexion échouera au lancement avec une erreur de module introuvable. Le nom de module à requérir est celui du pilote PostgreSQL — à vous de le trouver (indice : il apparaît dans le message d'erreur que Java vous affichera).

3.2 Stack conservée du Lab 2

Java 17+, JavaFX 21, Maven, FXML, CSS, Git/GitHub. **Rien ne change.** Les interdictions du Lab 2 restent en vigueur : pas de Swing, pas de `Collections.sort()` pour vos boutons de tri, pas de librairie de tri externe.

Vos tris restent obligatoires et doivent continuer à fonctionner. Il serait tentant de remplacer vos algorithmes par un `ORDER BY SQL`. **C'est interdit.** Vos tris codés à la main font partie de l'évaluation du Lab 2 *et* doivent toujours tourner au Lab 3. Le tri SQL est accepté uniquement en *plus*, jamais en remplacement.

4. Travail obligatoire — les 6 étapes

4.1 Étape 1 — Concevoir le schéma relationnel

Vous devez traduire votre modèle objet du Lab 2 en tables SQL.

- **Au minimum 2 tables** : votre entité principale, plus au moins une table liée par une **clé étrangère**
- Chaque table a une **clé primaire** explicite
- Les types SQL sont **choisis avec soin** : pas de TEXT partout, pas de prix en VARCHAR
- Au moins une **contrainte d'intégrité** au-delà des clés (NOT NULL, CHECK, UNIQUE)
- Le comportement de suppression est défini (ON DELETE CASCADE, SET NULL ou RESTRICT) et **vous devez pouvoir justifier votre choix** à l'oral

Exemple de découpage attendu selon les sujets (à adapter, ce ne sont que des pistes) :

Netflix	film et genre, ou film et réalisateur
Voitures	voiture et marque, ou voiture et vendeur
Spotify	chanson et artiste, ou chanson et album
NHL	joueur et équipe
Immobilier	propriété et quartier, ou propriété et courtier

Livrable : un fichier `schema.sql` versionné dans le dépôt, contenant les CREATE TABLE dans le bon ordre, et rejouable depuis une base vide.

4.2 Étape 2 — Peupler la base

Les 200 à 500 lignes de données de votre CSV doivent se retrouver dans PostgreSQL.

Deux approches acceptées, au choix :

1. Un fichier `donnees.sql` contenant les INSERT (généré depuis votre CSV par un petit script ou par un tableur)
2. La commande COPY de PostgreSQL, avec le `.csv` versionné dans le dépôt et la commande documentée dans le README

Le correcteur doit pouvoir recréer votre base entière à partir de ce que contient votre dépôt, sans vous contacter. Si votre base ne peut pas être reconstruite, la note de cette section est de zéro. **Testez-le** : supprimez votre base, rejouez vos scripts, relancez l'application.

4.3 Étape 3 — La classe de connexion

Une classe dédiée, unique, responsable de fournir une connexion à la base.

- Elle vit dans un package `[projet].dao` (ou `[projet].persistance`)
- Elle expose une méthode qui retourne une `Connection`
- Elle est la **seule** classe du projet à connaître l'URL JDBC

Les identifiants ne sont JAMAIS écrits en dur dans le code Java.

Ils vivent dans un fichier de configuration externe (`database.properties` par exemple), lu au démarrage. Ce fichier est **ajouté au .gitignore** et n'est **jamais** poussé sur GitHub — votre dépôt est public.

Vous versionnez à la place un fichier d'exemple (`database.properties.example`) avec des valeurs factices, pour que le correcteur sache quelles clés renseigner.

4.4 Étape 4 — L'interface DAO et son implémentation

Qu'est-ce que le pattern DAO ? *Data Access Object*. L'idée : isoler tout le code d'accès aux données derrière une interface, pour que le reste de l'application ignore *où* et *comment* les données sont stockées.

Votre service de filtrage ne doit pas savoir s'il parle à un CSV, à PostgreSQL ou à un fichier texte. Il demande des objets, il en reçoit.

Vous définissez une **interface** décrivant les opérations disponibles sur votre entité principale, puis une **implémentation** PostgreSQL de cette interface. Le contrat minimal attendu :

Opération	Rôle	SQL
trouverTous	Récupère toutes les entités de la table	SELECT
trouverParId	Récupère une entité précise, ou rien si absente	SELECT ... WHERE
ajouter	Insère une nouvelle entité et récupère son identifiant généré	INSERT
modifier	Met à jour une entité existante	UPDATE
supprimer	Supprime une entité par son identifiant	DELETE

- Les noms de méthodes sont libres, mais **cohérents** avec la langue de votre projet
- `trouverParId` doit gérer le cas « absent » proprement — un `Optional` est le choix attendu, pas un `null`
- **Aucune classe hors du package DAO n'importe** `java.sql.*`. C'est le critère qui prouve que votre isolation fonctionne.

Toute requête contenant une valeur variable utilise un PreparedStatement.

La concaténation de chaînes pour construire du SQL ("`... WHERE nom = ' + saisie + '`") est **interdite** et coûte des points, même si elle fonctionne. C'est la faille d'**injection SQL**, et vous devrez être capables d'expliquer à l'oral pourquoi elle est dangereuse.

4.5 Étape 5 — Brancher l'application sur le DAO

C'est ici que la qualité de votre Lab 2 se paye — en bien ou en mal.

- L'application charge désormais ses données **depuis PostgreSQL** au démarrage
- **Toutes les fonctionnalités du Lab 2 continuent de fonctionner** : tris, pagination, filtres, recherche, benchmark
- Vos contrôleurs ne changent pas (ou très peu) : ils continuent d'appeler vos services
- Votre `LecteurCSV` reste **dans le projet** : les deux implémentations coexistent derrière la même interface

Le critère de réussite de cette étape. On doit pouvoir passer du mode CSV au mode base de données en modifiant **une seule ligne** de votre code (celle qui choisit l'implémentation à instancier). Si vous devez toucher à vos contrôleurs, à vos services ou à vos tris, c'est que l'abstraction n'est pas au bon endroit.

4.6 Étape 6 — Le CRUD depuis l'interface graphique

Jusqu'ici votre application était en lecture seule. Elle devient modifiable.

- Un bouton **Ajouter** qui ouvre un formulaire de saisie

- Un bouton **Modifier** qui pré-remplit le formulaire avec l'élément sélectionné
- Un bouton **Supprimer** avec une **confirmation** avant l'opération
- Le tableau se **rafraîchit** après chaque opération
- Les saisies sont **validées** avant d'atteindre la base : champs obligatoires remplis, nombres réellement numériques, valeurs dans les bornes acceptables
- Les erreurs sont affichées **dans l'interface** (une **Alert** suffit), jamais dans la console

Une exception SQL ne doit jamais faire planter l'application. Base éteinte, mot de passe erroné, contrainte violée, suppression d'une ligne référencée ailleurs : dans tous ces cas l'utilisateur voit un message compréhensible et l'application reste utilisable.

Le correcteur **éteindra votre serveur PostgreSQL** et relancera votre application pour vérifier ce point.

5. Structure de packages attendue

La structure du Lab 2 est conservée, avec un package supplémentaire :

Package	Contenu
[projet]	MainFx.java uniquement
[projet].model	Classes du domaine (inchangé)
[projet].service	Logique métier (inchangé)
[projet].algorithmes	Vos tris (inchangé)
[projet].controller	Contrôleurs JavaFX
[projet].util	Utilitaires, dont LecteurCSV
[projet].dao	NOUVEAU — connexion, interfaces DAO, implémentations

Les règles de séparation du Lab 2 restent intégralement applicables, avec une règle de plus :

Règle 5 — Le SQL ne sort pas du package dao.

Aucun `import java.sql.*` ailleurs. Aucune chaîne contenant **SELECT**, **INSERT**, **UPDATE** ou **DELETE** ailleurs. Si un contrôleur contient du SQL, la note d'architecture tombe.

6. Bonus (jusqu'à +10%)

À traiter **uniquement** si le tronc obligatoire est terminé.

Bonus	Valeur
Statistiques SQL — un écran affichant des agrégats calculés par la base (COUNT , AVG , MIN , MAX avec GROUP BY)	+3%
Chargement asynchrone — les données se chargent dans une Task JavaFX, avec un indicateur de progression, sans figer l'interface	+3%
Authentification — table utilisateur , écran de connexion, mots de passe hachés (jamais en clair)	+4%
Favoris persistants — une table de liaison entre utilisateur et entité principale, avec ajout/retrait depuis l'interface	+3%
Graphiques — un BarChart ou PieChart JavaFX alimenté par une requête d'agrégation	+2%
Transactions — une opération multi-tables encadrée par commit / rollback , avec démonstration du rollback	+3%

Le total des bonus est plafonné à +10% de la note finale.

7. Présentation orale

7.1 Dates et inscription

Les présentations se déroulent sur **deux journées** :

Date	Jour
21 septembre 2026	Lundi
23 septembre 2026	Mercredi

Choix de votre date — action requise cette semaine.

Chaque équipe choisit sa journée de passage et l'inscrit sur le canal Teams du groupe. Les places de chaque journée sont **limitées** et attribuées dans l'ordre d'inscription.

L'horaire précis de chaque équipe sera **confirmé par le professeur** une fois toutes les inscriptions reçues. Les équipes ne s'étant pas inscrites se verront attribuer une date d'office.

7.2 Format

- **10 minutes** de présentation, suivies de **5 minutes** de questions
- **Tous les membres de l'équipe prennent la parole** — un membre silencieux perd les points de présentation
- Démonstration **en direct** de l'application qui tourne, connectée à la base
- Aucun diaporama obligatoire ; montrer le code et l'application suffit

7.3 Ce que vous devez montrer

1. Votre **schéma de base** dans pgAdmin : les tables, les clés, les relations
2. L'application qui **démarre et charge depuis PostgreSQL**
3. Une **opération CRUD complète** en direct : ajouter un élément, le voir apparaître, le modifier, le supprimer
4. Votre **interface DAO** et son implémentation, dans l'éditeur
5. La **ligne unique** qui permet de basculer entre CSV et base de données
6. Vos bonus, s'il y en a

7.4 Questions auxquelles vous devez savoir répondre

- Pourquoi avoir choisi ce découpage en tables plutôt qu'une seule grande table ?
- Pourquoi ce comportement **ON DELETE** sur votre clé étrangère ?
- Qu'est-ce qu'une injection SQL, et qu'est-ce qui protège votre code ?
- Que se passe-t-il dans votre application si la base est inaccessible ?
- Quelle partie de votre code aurait-il fallu modifier si vous n'aviez pas eu d'interface ?
- Qui a écrit quoi dans l'équipe ?

Les questions portent sur **votre** code. Un membre incapable d'expliquer une partie du projet qu'il est censé avoir écrite verra sa note individuelle ajustée à la baisse.

8. Remise

8.1 Le code — dimanche 20 septembre 2026, 23h59

Le représentant de l'équipe publie sur le canal Teams du groupe un `README.md` mis à jour contenant :

1. Les noms des membres de l'équipe et le sujet
2. Le lien vers le dépôt **GitHub** public (le même qu'au Lab 2)
3. La **procédure complète d'installation de la base** : créer la base, jouer `schema.sql`, jouer `donnees.sql`
4. Les clés attendues dans le fichier de configuration
5. La liste des fonctionnalités ajoutées au Lab 3, et des bonus traités
6. La répartition du travail entre les membres
7. La journée de présentation choisie

8.2 Ce que le dépôt doit contenir

- ✓ Tout le code source, dont le nouveau package `dao`
- ✓ `schema.sql` et `donnees.sql` (ou le CSV et la commande `COPY` documentée)
- ✓ `database.properties.example` avec des valeurs factices
- ✓ Le `pom.xml` à jour avec le pilote JDBC
- ✓ Le `README.md` mis à jour
- ✗ **Jamais** le vrai `database.properties` avec vos mots de passe
- ✗ Pas de `target/`, `.idea/`, `out/`

Vérification de sécurité. Votre dépôt est public. Avant de pousser, relisez votre `.gitignore` et faites une recherche de votre mot de passe dans l'historique Git. Un mot de passe poussé par erreur reste dans l'historique même après suppression du fichier : il faut alors le changer sur le serveur.

8.3 Politique de retard

- Retard de 1 jour : pénalité de 10%
- Retard de 2 jours : pénalité de 25%
- Retard de 3 jours et plus : note de 0

Les présentations ayant lieu le lendemain de la remise, un retard vous fait présenter un projet déjà pénalisé. **La date de présentation n'est pas repoussable.**

9. Barème de correction (100 points)

Critère	Points
Interface DAO + implémentation PostgreSQL (CRUD complet)	25
Schéma SQL : tables, clés, types, contraintes, scripts rejouables	15
Intégration dans l'application existante sans régression (Lab 2 toujours fonctionnel)	15
Opérations CRUD depuis l'interface graphique, avec validation des saisies	15
Robustesse et sécurité : <code>PreparedStatement</code> , <code>try-with-resources</code> , gestion des <code>SQLException</code>	10
Classe de connexion isolée + configuration externalisée hors du dépôt	10
Git (commits répartis et réguliers) + README complet et exact	5
Présentation orale (clarté, maîtrise du code, participation de tous)	5
TOTAL	100

Points bonus : jusqu'à +10% selon la section 6.

Conditions éliminatoires — dans ces cas, la note est plafonnée à 50 :

- Le projet ne compile pas ou ne démarre pas
- La base ne peut pas être reconstruite à partir du dépôt
- Du SQL se trouve en dehors du package `dao`
- Les identifiants de connexion sont en dur dans le code ou poussés sur GitHub

10. Organisation suggérée sur la semaine

10.1 Répartition des rôles

Équipe de 2 :

- Personne A : schéma SQL, scripts, connexion, implémentation DAO
- Personne B : formulaires JavaFX, validation des saisies, gestion des erreurs, README

Équipe de 3 :

- Personne A : schéma SQL, scripts de création et de peuplement
- Personne B : connexion, interface DAO et implémentation PostgreSQL
- Personne C : formulaires CRUD, validation, alertes, README et préparation de l'oral

Les personnes B et C travaillent en parallèle si B définit **l'interface DAO en premier** et la pousse immédiatement. C peut alors coder ses formulaires contre cette interface pendant que B l'implémente. C'est exactement à cela que servent les interfaces dans une équipe.

10.2 Calendrier conseillé

Jour	Objectif
Lundi 14	PostgreSQL installé chez tous les membres. Choix de la journée de présentation inscrit sur Teams.
Mardi 15	<code>schema.sql</code> écrit et joué. Base créée, tables visibles dans pgAdmin.
Mercredi 16	Données importées. Connexion Java qui fonctionne (un simple <code>SELECT</code> affiché suffit).
Jeudi 17	Interface DAO définie, <code>trouverTous</code> et <code>trouverParId</code> implémentés.
Vendredi 18	Application branchée sur le DAO. Tous les tris, filtres et pagination revérifiés.
Samedi 19	<code>ajouter</code> , <code>modifier</code> , <code>supprimer</code> et les formulaires. Gestion des erreurs.
Dimanche 20	Tests finaux, README, remise. Répétition de la présentation.

Testez la reconstruction complète le samedi, pas le dimanche soir. Supprimez la base, rejouez vos scripts, relancez l'application depuis un clone frais du dépôt. C'est le test que fera le correcteur, et c'est là que les surprises apparaissent.

11. Questions fréquentes

On n'a pas fait d'interface `SourceDonnees` au Lab 2. On fait quoi ?

Vous commencez par l'extraire. Créez l'interface, faites-la implémenter par votre `LecteurCSV` existant, vérifiez que l'application fonctionne toujours, **puis** écrivez l'implémentation PostgreSQL. C'est un refactoring d'une heure environ, et il vous fera gagner du temps ensuite.

Peut-on utiliser Hibernate, JPA ou Spring Data ?

Non. L'objectif est précisément de comprendre ce que ces outils font à votre place. Vous écrivez le JDBC à la main.

Peut-on remplacer nos tris Java par `ORDER BY` ?

Non. Vos tris codés à la main doivent toujours fonctionner. Un `ORDER BY` peut s'ajouter en complément, jamais en remplacement.

Faut-il garder le fichier CSV ?

Oui. Le CSV et son lecteur restent dans le projet. Les deux implémentations coexistent derrière la même interface — c'est même la démonstration que votre architecture est correcte.

Peut-on utiliser une autre base (MySQL, SQLite, MongoDB) ?

Non, PostgreSQL est imposé : c'est celle vue en séance 13, et celle que le correcteur installera.

Notre base est sur l'ordinateur d'un seul membre. C'est un problème ?

Oui. **Chaque membre** doit pouvoir recréer la base localement à partir des scripts du dépôt. C'est justement pour cela que les scripts sont versionnés.

Doit-on changer d'équipe ou de sujet ?

Non. Même équipe, même sujet, même dépôt qu'au Lab 2. En cas de difficulté d'équipe, contactez le professeur **avant** le vendredi 18 septembre.

Les outils d'IA sont-ils permis ?

Même règle qu'au Lab 2 : acceptés pour comprendre un concept ou déboguer une erreur, refusés pour générer votre code. À l'oral, vous devrez expliquer chaque ligne de ce que vous avez remis.

Bon développement !

Remise dimanche 20 septembre, 23h59 — présentations lundi 21 et mercredi 23 septembre